TOPIC 5.2

Feature Selection Techniques

Choosing the columns that matter: filter, wrapper and embedded methods

WHAT YOU WILL LEARN

- Explain **why** we select features and what goes wrong when we keep every column
- Name the three families of methods: **filter, wrapper and embedded**, and how they differ
- Carry out a **chi-square test** by hand, and compute **information gain** and **correlation**
- Describe **recursive feature elimination (RFE)**, forward selection and genetic algorithms
- Use a **decision tree** and **Lasso** as built-in feature selectors
- Compare the methods on a real data set and explain why they do not always agree

1. Why select features?

A data set is a table. Each **column** is a **feature** (an input), and one special column is the **target** (what we want to predict). In the class whiteboard example the table has features **A, B, C, D, E** and a target **T**.

It is tempting to give the model every column. That usually hurts:

- **Overfitting.** Columns that carry no real information give the model extra freedom to memorise noise, so it does well on training data and worse on new data.
- **The curse of dimensionality.** With many columns, the rows no longer cover all the combinations of values, so the data is thin everywhere.
- **Cost.** More columns mean more memory and slower training and prediction.
- **Clarity.** A model with 4 inputs is far easier to explain than one with 40.

DEFINITION **Feature selection**

Choosing a **subset of the existing columns** that gives the best model, and discarding the rest. (This is different from *feature extraction*, such as PCA, which builds **new** columns out of the old ones.)

2. The three families

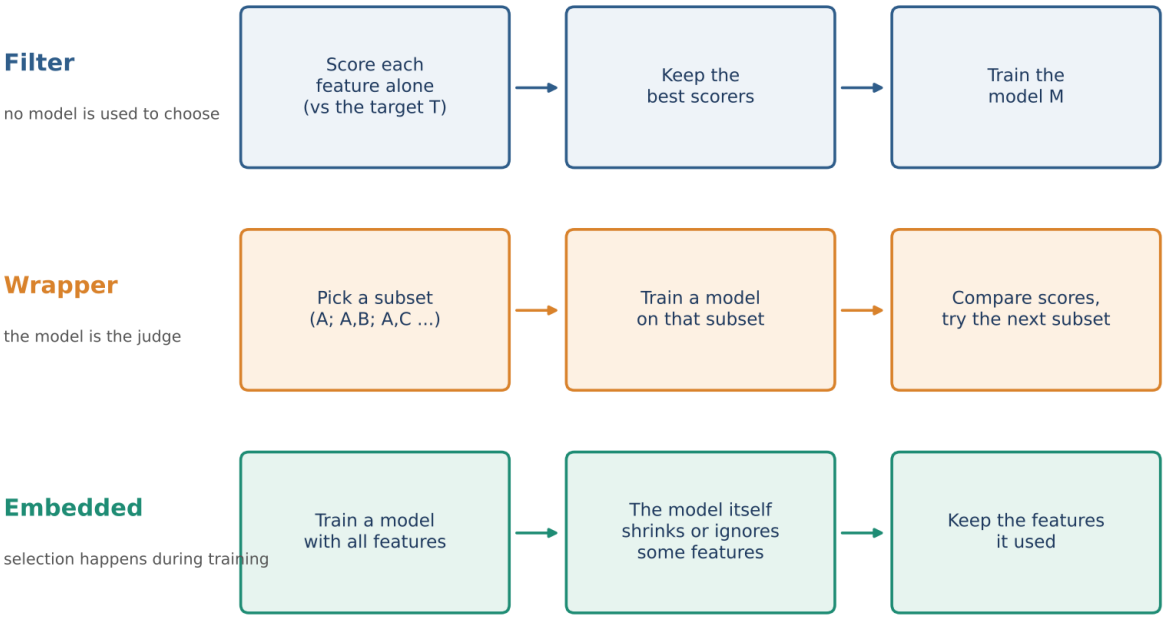


Figure 1. How the three families work. They differ in *when* and *how* the choice is made.

	Filter	Wrapper	Embedded
Idea	Score each feature on its own, keep the best	Try subsets, keep the one whose model scores best	The model picks features while it trains
Uses a model to decide?	No	Yes, many times	Yes, once
Speed	Fastest	Slowest	Fast
Sees feature teamwork?	No (each feature alone)	Yes	Yes
Class examples	Information gain, chi-square test, correlation	Recursive feature elimination, genetic algorithms	Decision tree

3. Filter methods

A filter method looks at the **data only**: it scores every feature by how strongly it is linked to the target, then keeps the highest scores. It is quick and does not depend on any model.

3.1 Correlation coefficient

For two numeric columns, the correlation coefficient **r** runs from −1 to +1. Values near +1 or −1 mean a strong straight-line link with the target; near 0 means none. We keep features with a large **|r|** (the absolute value, because −0.8 is as informative as +0.8).

On the class car data (predicting `mpg`, fuel economy), weight has r = −0.832 and engine size `disp` has −0.804, so heavier cars with bigger engines do fewer miles per gallon. Acceleration is weak at 0.420.

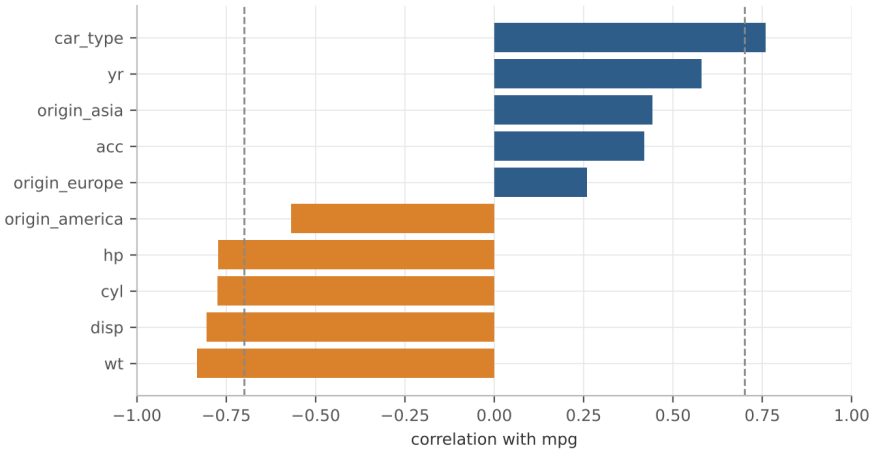


Figure 2. Correlation of each feature with mpg. The dashed lines mark |r| = 0.7, one possible cut-off.

LIMITATION **A filter's blind spot**

Each feature is scored **alone**, so the filter cannot see that features overlap. Here `cyl`, `disp`, `hp` and `wt` are strongly correlated with *each other* (for example cyl and disp: 0.95). A correlation filter would keep all four, even though together they say nearly the same thing.

3.2 Chi-square test

The chi-square (χ^2) test compares what we **observed** with what we would **expect** if nothing were going on:

$$\chi^2 = \sum_i \frac{(O_i - E_i)^2}{E_i}$$

A big χ^2 means observed counts are far from expected, so there is a real pattern. We decide by comparing it with a **critical value** from the χ^2 table, which depends on the **degrees of freedom (df)** and the significance level (usually 5%). If χ^2 is **larger** than the critical value, the pattern is real. If it is smaller, it could be chance.

WORKED EXAMPLE **Class example: are aircraft accidents spread evenly over the week?**

- Accidents Mon to Sun: 14, 18, 12, 11, 15, 14, 14. Total 98. If evenly spread, each day expects 98 / 7 = **14**
- $(O - E)^2 / E$ per day: 0, 1.14, 0.29, 0.64, 0.07, 0, 0 (that is, 0, 16/14, 4/14, 9/14, 1/14, 0, 0)
- χ^2 = 30 / 14 = **2.14**. Degrees of freedom = 7 − 1 = **6**; the critical value at 5% is **12.592**
- 2.14 < 12.592, so the hypothesis is accepted: the accident counts are consistent with an even spread (the differences are just chance)

The class example tests one column against a fixed expectation. For **feature selection** we use the same idea in a different way: does the feature's value change how the **target** turns out? Take a table of counts of a categorical feature against a categorical target. Under *independence* (the feature tells us nothing), the expected count in each cell is (row total × column total) / grand total.

WORKED EXAMPLE **Does revising before the exam relate to passing?**

- 100 students. Revised: 40 passed, 10 failed. Did not revise: 20 passed, 30 failed. Overall 60 passed and 40 failed
- If revision made no difference, 50 who revised would give 30 passes and 20 fails, and likewise for the other 50: **expected 30, 20, 30, 20**
- $\chi^2 = 10^2/30 + 10^2/20 + 10^2/30 + 10^2/20 =$ **16.67**. Degrees of freedom = $(2-1)(2-1) = 1$; critical value **3.841**
- 16.67 > 3.841, so revision and passing are **related**: **keep** this feature
- A second feature that splits the students 30 pass / 20 fail in **both** groups gives $\chi^2 = 0$, so it tells us nothing about the result: **drop** it

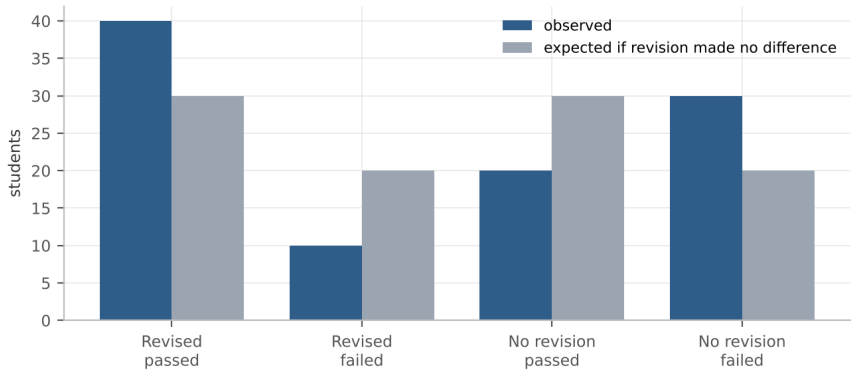


Figure 3. For the revision example, the observed counts (blue) differ clearly from the counts expected if revision did not matter (grey).

The chi-square test needs **categorical** features and a categorical target (classification). In scikit-learn, `SelectKBest(chi2, k=...)` applies this idea; it requires non-negative feature values such as counts.

3.3 Information gain

Information gain measures **how much a feature reduces uncertainty** about the target. Uncertainty is measured by **entropy**; for a target with class probabilities p it is:

$$H = -\sum_i p_i \log_2 p_i$$

Entropy is 0 when the outcome is certain and largest when the classes are evenly mixed. **Information gain = entropy of the target − the average entropy after splitting by the feature.**

WORKED EXAMPLE **Information gain of the revision feature**

- Target: 60% pass, 40% fail. Entropy = $-0.6 \cdot \log_2 0.6 - 0.4 \cdot \log_2 0.4 =$ **0.9710**
- Revised group (40 pass, 10 fail): entropy **0.7219**. Did-not-revise group (20 pass, 30 fail): entropy **0.9710**
- Average after the split (each group is half the students) = 0.5 × 0.7219 + 0.5 × 0.9710 = 0.8464
- **Information gain = 0.9710 − 0.8464 = 0.1245**. The useless second feature would give a gain of 0

Decision trees use information gain (or a close relative) to choose where to split, which links filters to the embedded methods below. For numeric targets, scikit-learn's `mutual_info_regression` plays a similar role.

4. Wrapper methods

A wrapper method **wraps a model around the search**: it trains the model on different subsets of features and keeps the subset with the best score. It sees how features work **together**, but it trains many models, so it is slow.

4.1 Recursive feature elimination (RFE)

1. Train the model on **all** features
2. Rank the features by importance (for a linear model, by the size of the coefficients)
3. **Remove the weakest** one
4. Repeat on the remaining features until the number you want is left

It is *recursive* because the model is retrained after every removal, so importance is re-judged each time.

4.2 Forward selection and backward elimination

The class whiteboard tried subsets of the columns A, B, C, ... with a model on each (M_1, M_2, M_3, M_4). The systematic version of that is **forward selection**: start with no features, add the single feature that improves the model most, then the next, and stop when nothing helps. **Backward elimination** goes the other way, from all features downwards. Both are greedy, which means they take the best step at each moment without looking further ahead.

4.3 Genetic algorithms

A genetic algorithm borrows from evolution to search through the huge number of possible subsets (with 20 features there are over a million):

1. Start with a **population** of random subsets (each is a string of keep or drop for every feature)
2. Train a model on each subset; its score is its **fitness**
3. Keep the fittest, and create new subsets by **crossover** (mixing two parents) and **mutation** (random flips)
4. Repeat for many generations

It can find good combinations that greedy methods miss, at the cost of a lot of computation.

5. Embedded methods

In an embedded method, selection is a **by-product of training**. You train one model and read off which features it used.

- **Decision tree.** At each split the tree picks the feature that separates the data best. A feature's *importance* is the total improvement it contributed over the whole tree. Features never used have importance 0.
- **Lasso regression.** Adds a penalty that pushes the coefficients of weak features to exactly 0 (see the Regularization notes, 5.5).

6. All of them on one data set

Using the class car data (398 cars, 10 features after turning `origin` into three 0/1 columns), the table shows which features each method picks. Every threshold is a choice, so these are examples rather than rules.

Method	cyl	disp	hp	wt	acc	yr	car_type	USA	Asia	Eur.
Correlation ($\|r\| \geq 0.7$)	✓	✓	✓	✓	–	–	✓	–	–	–
Mutual information (top 4)	✓	✓	✓	✓	–	–	–	–	–	–
RFE (keep 4)	✓	–	–	✓	–	✓	✓	–	–	–
Decision tree (importance > 0.05)	✓	–	✓	✓	–	✓	–	–	–	–
Lasso (coefficient ≠ 0)	–	–	✓	✓	–	✓	✓	✓	–	–

The last three columns are the three origin dummies (USA, Asia, Europe). The methods **do not fully agree**, and that is normal. `wt` is picked by every method. `cyl`, `disp` and `hp` are nearly interchangeable (they are correlated with each other at 0.84 to 0.95), so one method might favour `cyl` (the tree gives it importance 0.625), another `disp` or `hp`, and Lasso drops `cyl` and `disp` altogether because `wt` and `hp` already carry their information.

WORKED EXAMPLE **Does dropping columns hurt?**

- A linear model on all 10 features scores R^2 = **0.8439** on the test cars
- A linear model on only 4 features (`wt`, `yr`, `hp`, `car_type`) scores **0.8267**
- Six fewer columns cost less than 0.02 of R^2, and the model is simpler to explain and faster to train

7. Good practice

- **Select features using the training data only.** If you choose features after looking at the test rows, information from the test set leaks into your model and the test score looks better than it should.
- **Combine methods and common sense.** Remove obvious junk first (ID columns, constants), then use a filter to cut the list, then a wrapper or embedded method to fine-tune.
- **Use domain knowledge.** A feature that makes sense for the problem deserves a second look before you drop it.
- **Check the result.** Compare the model's score with and without the chosen subset, ideally with cross-validation.

Summary

SUMMARY

- Feature selection keeps the useful columns to reduce overfitting, cost and complexity
- **Filter** methods (information gain, chi-square, correlation) score features **without a model**: fast, but blind to feature teamwork
- **Wrapper** methods (RFE, forward and backward selection, genetic algorithms) **search with a model**: accurate but slow
- **Embedded** methods (decision tree, Lasso) select **while training**: a good balance
- $\chi^2 = \Sigma\ (O - E)^2 / E$: compare it with the critical value. Information gain = entropy before − entropy after the split
- Different methods can pick different features when the features are correlated; select on training data only

Practice questions

Try each question before reading the answer underneath it.

Q1. What is the difference between feature selection and feature extraction?

Answer. Selection **keeps some of the original columns** and drops the rest. Extraction (for example PCA) **creates new columns** from the old ones.

Q2. Why can a filter method keep two features that say the same thing?

Answer. It scores each feature on its own against the target. It never checks whether two features are strongly correlated with *each other*.

Q3. In the accident example $\chi^2 = 2.14$ and the critical value is 12.592. What do you conclude?

Answer. χ^2 is smaller than the critical value, so the differences between days are consistent with chance: accidents appear to be evenly spread.

Q4. A split by feature F leaves the target as uncertain as before. What is its information gain?

Answer. **Zero**. Information gain is the reduction in entropy, and there is no reduction.

Q5. Why is a wrapper method slower than a filter method?

Answer. It must **train and score a model for every subset** it tries, while a filter scores each feature once without training a model.

Q6. Why should feature selection be done on the training set only?

Answer. Choosing features with the test rows in view leaks test information into the model, so the test score is no longer an honest estimate of performance on new data.